

INTEGRANTES DO GRUPO:

- 1) Davi Barcia
- 2) Daniel Rozendo
- 3) Slann Antonio
- 4) Renan Marinho
- 5) Ismael Victor

CONVENÇÕES ADOTADAS NA CONTAGEM

1. Cada linha de comando simples (atribuição, comparação, incremento, retorno) custa 1 instrução.
2. Um laço "para i <- 0 até n-1" executa o teste n+1 vezes (n testes que dão VERDADEIRO e 1 teste que dá FALSO, encerrando o laço). O corpo executa n vezes.
3. Em um "se/senão", considera-se sempre o PIOR CASO, ou seja, o ramo mais caro.
4. O computador de referência executa 10^8 instruções por segundo, logo:
 $\text{Tempo} = f(n) / 10^8$ (em segundos)
5. As funções auxiliares de criação, preenchimento e impressão dos arranjos não entram na análise (conforme o enunciado).

FUNÇÃO 1 - CONTAGEM DE OCORRÊNCIAS DISTINTAS

PSEUDOCÓDIGO

CUSTO DA LINHA

01	função contarOcorrencias(V[], n, B[], k)	
02	soma <- 0	1
03	para i <- 0 até k-1 faça	k + 1
04	cont <- 0	k
05	para j <- 0 até n-1 faça	k * (n + 1)
06	se (V[j] = B[i]) então	k * n
07	cont <- cont + 1	k * n
08	fim se	
09	fim para	
10	soma <- soma + cont	k
11	fim para	
12	retorne soma	1
13	fim função	

EXPLICAÇÃO DAS LINHAS MAIS IMPORTANTES:

- Linha 03: o laço externo roda k vezes, então são k testes verdadeiros + 1 falso.
- Linha 05: para CADA uma das k voltas do laço externo, o laço interno faz (n + 1) testes. Por isso $k * (n + 1)$.
- Linhas 06 e 07: o corpo do laço interno executa n vezes para cada uma das k voltas externas = $k * n$. A linha 07 é contada no PIOR CASO, que é quando todos os n elementos do vetor são iguais ao elemento buscado.

EXPRESSÃO DE COMPLEXIDADE:

$$f(n,k) = 1 + (k + 1) + k + k(n + 1) + k*n + k*n + k + 1$$

$$f(n,k) = 1 + k + 1 + k + k*n + k + k*n + k*n + k + 1$$

$$f(n,k) = 3kn + 4k + 3$$

BIG O:

$$O(n * k)$$

(O termo dominante é $3kn$; descartando constantes e termos de menor ordem, a complexidade é quadrática em relação ao produto das duas entradas. Se k fosse da mesma ordem de n , seria $O(n^2)$.)

CÁLCULO DE TEMPO ($n = 50.000$ e $k = 4.000$, a 10^8 instruções/segundo):

$$f(50000, 4000) = 3 * 4000 * 50000 + 4 * 4000 + 3$$

$$= 600.000.000 + 16.000 + 3$$

$$= 600.016.003 \text{ instruções}$$

$$\text{Tempo} = 600.016.003 / 10^8 = 6,00016003 \text{ segundos}$$

FUNÇÃO 2 - ANÁLISE DE PARES EM MATRIZ TRIANGULAR

PSEUDOCÓDIGO	CUSTO DA LINHA

01 função analisarPares(A[], n)	
02 cont <- 0	1
03 para i <- 0 até n-1 faça	n + 1
04 para j <- i até n-1 faça	n(n+1)/2 + n
05 se ((A[i][j] + A[j][i]) mod 5 = 0) então	n(n+1)/2
06 cont <- cont + 1	n(n+1)/2
07 fim se	
08 fim para	
09 fim para	
10 retorne cont	1
11 fim função	

EXPLICAÇÃO DO LAÇO INTERNO (o ponto central desta questão):

O laço interno começa em $j = i$ (e não em $j = 0$), porque só precisamos tratar a DIAGONAL PRINCIPAL e a METADE SUPERIOR da matriz. Logo o número de repetições NÃO é regular: ele diminui a cada volta do laço externo.

$i = 0 \rightarrow j$ vai de 0 até $n-1 \rightarrow n$ repetições
 $i = 1 \rightarrow j$ vai de 1 até $n-1 \rightarrow n-1$ repetições
 $i = 2 \rightarrow j$ vai de 2 até $n-1 \rightarrow n-2$ repetições
...
 $i = n-1 \rightarrow j$ vai de $n-1$ até $n-1 \rightarrow 1$ repetição

Somando, temos a soma dos n primeiros números naturais (Progressão Aritmética):

$$n + (n-1) + (n-2) + \dots + 2 + 1 = n(n + 1) / 2$$

Esse é o número de vezes que o CORPO do laço interno executa (linhas 05 e 06). Já o TESTE da linha 04 executa uma vez a mais em cada volta do laço externo (o teste que dá falso), por isso:

$$\text{Linha 04} = [n(n+1)/2] + n$$

EXPRESSÃO DE COMPLEXIDADE:

$$f(n) = 1 + (n + 1) + [n(n+1)/2 + n] + n(n+1)/2 + n(n+1)/2 + 1$$

$$f(n) = 3 * [n(n+1)/2] + 2n + 3$$

$$f(n) = (3n^2 + 3n)/2 + 2n + 3$$

$$f(n) = 1,5n^2 + 1,5n + 2n + 3$$

$$f(n) = 1,5n^2 + 3,5n + 3$$

BIG O:

$$O(n^2)$$

(Mesmo percorrendo apenas metade da matriz, a constante 1/2 é descartada na notação assintótica: $n(n+1)/2$ continua sendo $O(n^2)$.)

CÁLCULO DE TEMPO ($n = 500$, a 10^8 instruções/segundo):

$$f(500) = 1,5 * 500^2 + 3,5 * 500 + 3$$

$$= 1,5 * 250.000 + 1.750 + 3$$

$$= 375.000 + 1.750 + 3$$

$$= 376.753 \text{ instruções}$$

$$\text{Tempo} = 376.753 / 10^8 = 0,00376753 \text{ segundos}$$

FUNÇÃO 3 - COMPARAÇÃO DE MATRIZES TRIDIMENSIONAIS

PSEUDOCÓDIGO	CUSTO DA LINHA

01 função compararArranjos(A[][][], B[][][], n)	
02 somaA <- 0	1
03 para i <- 0 até n-1 faça	n + 1
04 para j <- 0 até n-1 faça	n(n + 1)
05 para k <- 0 até n-1 faça	n^2 (n + 1)
06 somaA <- somaA + A[i][j][k]	n^3
07 fim para	
08 fim para	
09 fim para	
10 somaB <- 0	1
11 para i <- 0 até n-1 faça	n + 1
12 para j <- 0 até n-1 faça	n(n + 1)
13 para k <- 0 até n-1 faça	n^2 (n + 1)
14 somaB <- somaB + B[i][j][k]	n^3
15 fim para	
16 fim para	
17 fim para	

18	se (somaA >= somaB) então	1
19	retorne 1	1
20	senão	
21	retorne 0	(pior caso já contado)
22	fim se	
23	fim função	

EXPLICAÇÃO:

- Linha 04: o teste do segundo laço é executado $(n+1)$ vezes para cada uma das n voltas do primeiro laço = $n(n+1)$.
- Linha 05: o teste do terceiro laço é executado $(n+1)$ vezes para cada uma das n^2 combinações de i e j = $n^2(n+1)$.
- Linha 06: o comando mais interno executa $n * n * n = n^3$ vezes.
- Os dois blocos (arranjo A e arranjo B) são SEQUENCIAIS, e não aninhados. Por isso os custos se SOMAM ($2 * n^3$) em vez de se multiplicarem.

EXPRESSÃO DE COMPLEXIDADE:

$$\begin{aligned}
 \text{Custo de um bloco} &= 1 + (n+1) + n(n+1) + n^2(n+1) + n^3 \\
 &= 1 + n + 1 + n^2 + n + n^3 + n^2 + n^3 \\
 &= 2n^3 + 2n^2 + 2n + 2
 \end{aligned}$$

$$\begin{aligned}
 f(n) &= 2 * (2n^3 + 2n^2 + 2n + 2) + 1 + 1 \\
 f(n) &= 4n^3 + 4n^2 + 4n + 6
 \end{aligned}$$

BIG O:

$$O(n^3)$$

CÁLCULO DE TEMPO ($n = 300$, a 10^8 instruções/segundo):

$$\begin{aligned}
 f(300) &= 4 * 300^3 + 4 * 300^2 + 4 * 300 + 6 \\
 &= 4 * 27.000.000 + 4 * 90.000 + 1.200 + 6 \\
 &= 108.000.000 + 360.000 + 1.200 + 6 \\
 &= 108.361.206 \text{ instruções}
 \end{aligned}$$

$$\text{Tempo} = 108.361.206 / 10^8 = 1,08361206 \text{ segundos}$$

FUNÇÃO 4 - ANÁLISE DE CASOS ASSIMÉTRICOS NO CONDICIONAL

FUNÇÃO AUXILIAR (necessária para a análise do ramo "senão"):

PSEUDOCÓDIGO	CUSTO DA LINHA
01 função fatorial(x)	
02 f <- 1	1
03 para i <- 2 até x faça	x
04 f <- f * i	x - 1
05 fim para	
06 retorne f	1

07 fim função

$$g(x) = 1 + x + (x - 1) + 1 = 2x + 1 \quad \rightarrow \quad O(x)$$

(O laço vai de 2 até x, ou seja, x-1 repetições do corpo e x testes, contando o teste final que dá falso.)

FUNÇÃO PRINCIPAL:

PSEUDOCÓDIGO	CUSTO DA LINHA
01 função processar_vetor(V[], n)	
02 soma <- 0	1
03 para i <- 0 até n-1 faça	n + 1
04 se (V[i] mod 2 = 0) então	n
05 soma <- soma + V[i]	custo 1 (ramo PAR)
06 senão	
07 soma <- soma + fatorial(V[i])	custo 2m + 2 (ramo ÍMPAR)
08 fim se	
09 fim para	
10 retorne soma	1
11 fim função	

EXPLICAÇÃO DO CONDICIONAL ASSIMÉTRICO (o ponto central desta questão):
Os dois ramos do "se" têm custos MUITO diferentes:

- Ramo PAR (linha 05): custo 1 $\rightarrow$ constante, $O(1)$
- Ramo ÍMPAR (linha 07): custo da chamada do fatorial, $g(m) = 2m + 1$, mais 1 instrução da soma/atribuição = $2m + 2$, sendo m o valor guardado em V[i].
Ou seja, esse ramo é $O(m)$, linear no VALOR do elemento.

Como a análise é sempre feita pelo PIOR CASO, consideramos que:

- (a) TODOS os n elementos do vetor são ÍMPARES (o ramo caro executa n vezes); e
- (b) m é o maior valor armazenado no vetor.

$$\begin{aligned}f(n,m) &= 1 + (n + 1) + n + n(2m + 2) + 1 \\f(n,m) &= 2mn + 2n + 2n + 3 \\f(n,m) &= 2mn + 4n + 3 \quad \rightarrow \quad O(n * m)\end{aligned}$$

Adotando a convenção usada em sala de que, no pior caso, os valores guardados no vetor são da mesma ordem de grandeza do tamanho do vetor ($m = n$), temos:

EXPRESSÃO DE COMPLEXIDADE:

$$f(n) = 2n^2 + 4n + 3$$

BIG O:

$$O(n^2)$$

(Se os valores do vetor fossem limitados por uma constante C, o custo do fatorial seria constante e a função seria apenas $O(n)$. É justamente a assimetria do condicional que faz a complexidade saltar de $O(n)$ para $O(n^2)$.)

CÁLCULO DE TEMPO ($n = 50.000$ no pior caso, a 10^8 instruções/segundo):

$$\begin{aligned} f(50000) &= 2 * 50.000^2 + 4 * 50.000 + 3 \\ &= 2 * 2.500.000.000 + 200.000 + 3 \\ &= 5.000.000.000 + 200.000 + 3 \\ &= 5.000.200.003 \text{ instruções} \end{aligned}$$

$$\text{Tempo} = 5.000.200.003 / 10^8 = 50,00200003 \text{ segundos}$$

FUNÇÃO 5 - CONTAGEM DE ELEMENTOS PRESENTES EM VETOR ORDENADO

FUNÇÃO AUXILIAR - BUSCA BINÁRIA:

PSEUDOCÓDIGO	CUSTO DA LINHA

01 função buscaBinaria(B[], n, x)	
02 ini <- 0	1
03 fim <- n - 1	1
04 enquanto (ini <= fim) faça	$\log_2(n) + 1$
05 meio <- (ini + fim) div 2	$\log_2(n)$
06 se (B[meio] = x) então	$\log_2(n)$
07 retorne 1	(sai da função)
08 senão se (B[meio] < x) então	$\log_2(n)$
09 ini <- meio + 1	$\log_2(n)$
10 senão	
11 fim <- meio - 1	(pior caso já contado)
12 fim se	
13 fim enquanto	
14 retorne 0	1
15 fim função	

EXPLICAÇÃO:

A cada volta do "enquanto", o trecho de busca é dividido pela metade: $n, n/2, n/4, n/8, \dots, 1$. O número de divisões por 2 até sobrar 1 elemento é exatamente $\log_2(n)$. O PIOR CASO da busca binária é o elemento NÃO existir no vetor (o laço vai até o fim, sem sair pelo "retorne 1" da linha 07).

$$\begin{aligned} b(n) &= 1 + 1 + (\log_2(n) + 1) + \log_2(n) + \log_2(n) + \log_2(n) + \log_2(n) + 1 \\ b(n) &= 5 * \log_2(n) + 4 \quad \rightarrow O(\log n) \end{aligned}$$

FUNÇÃO PRINCIPAL:

PSEUDOCÓDIGO	CUSTO DA LINHA

```

01 função contarPresentes(A[], B[], n)
02   total <- 0                                1
03   para i <- 0 até n-1 faça                    n + 1
04     total <- total + buscaBinaria(B, n, A[i])  n * (5*log2(n) + 5)
05   fim para
06   retorne total                                1
07 fim função

```

EXPLICAÇÃO DA LINHA 04:

Ela executa n vezes, e cada execução custa o preço da busca binária inteira $[b(n) = 5 \cdot \log_2(n) + 4]$ mais 1 instrução da soma/atribuição, ou seja, $5 \cdot \log_2(n) + 5$.

EXPRESSÃO DE COMPLEXIDADE:

$$f(n) = 1 + (n + 1) + n \cdot (5 \cdot \log_2(n) + 5) + 1$$

$$f(n) = 1 + n + 1 + 5n \cdot \log_2(n) + 5n + 1$$

$$f(n) = 5n \cdot \log_2(n) + 6n + 3$$

BIG O:

$$O(n \log n)$$

CÁLCULO DE TEMPO ($n = 10.000.000$, a 10^8 instruções/segundo):

$$\log_2(10.000.000) = 23,2535 \text{ (pois } 2^{23,2535} = 10.000.000)$$

$$f(10^7) = 5 \cdot 10.000.000 \cdot 23,2535 + 6 \cdot 10.000.000 + 3$$

$$= 50.000.000 \cdot 23,2535 + 60.000.000 + 3$$

$$= 1.162.674.833 + 60.000.000 + 3$$

$$= 1.222.674.836 \text{ instruções}$$

$$\text{Tempo} = 1.222.674.836 / 10^8 = 12,22674836 \text{ segundos}$$

RESUMO GERAL

FUNÇÃO	EXPRESSÃO DE COMPLEXIDADE	BIG O	ENTRADA	TEMPO
1	$3kn + 4k + 3$	$O(n \cdot k)$	$n=50.000 \text{ } k=4.000$	6,00016 s
2	$1,5n^2 + 3,5n + 3$	$O(n^2)$	$n=500$	0,0037675 s
3	$4n^3 + 4n^2 + 4n + 6$	$O(n^3)$	$n=300$	1,08361 s
4	$2n^2 + 4n + 3$ (pior caso)	$O(n^2)$	$n=50.000$	50,002 s
5	$5n \cdot \log_2(n) + 6n + 3$	$O(n \log n)$	$n=10.000.000$	12,2267 s

(Todos os tempos foram calculados para um computador que executa 10^8 instruções por segundo.)

OBSERVAÇÕES SOBRE O PROGRAMA

1. Todos os arranjos são alocados DINAMICAMENTE com malloc() e liberados com

free(), portanto qualquer dimensão informada pelo usuário funciona, inclusive os tamanhos pedidos nos cálculos de tempo.

2. A função criarEPreencherArranjoAleatorio() atende ao item 1 do tópico ATENÇÃO: ela pergunta quantas linhas e quantas colunas o arranjo deve ter, cria a matriz dinâmica e preenche tudo com valores aleatórios. Ela é usada pela Função 2 quando o usuário escolhe o preenchimento AUTOMÁTICO (como a Função 2 exige matriz quadrada, o programa pede novamente se linhas for diferente de colunas) e também pode ser executada sozinha pela opção 6 do menu principal.
3. Sempre que uma das 5 funções é escolhida, o programa pergunta se o preenchimento do arranjo deve ser MANUAL ou AUTOMÁTICO (aleatório), e em seguida imprime o arranjo criado na tela ANTES de mostrar o resultado da operação, para que se possa conferir se a função está correta.
4. Quando o arranjo tem mais de 200 elementos (caso dos tamanhos usados nos cálculos de tempo, como $n = 10.000.000$), o programa pergunta antes se o usuário realmente quer imprimir tudo na tela, evitando travar o terminal.
5. Além do resultado, cada função mostra o TEMPO REAL de execução medido com clock(), o que permite comparar a previsão teórica com a prática.
6. Na Função 4, no modo aleatório, os valores são sorteados entre 0 e 20 para que o cálculo do fatorial não estoure a capacidade do tipo numérico (unsigned long long suporta até 20!).
7. Na Função 5, o vetor B é ordenado automaticamente antes da execução, já que a busca binária só funciona em vetor ordenado (o enunciado informa que B já chega ordenado; a ordenação é apenas uma preparação dos dados de teste e, por isso, não entra na análise de complexidade).
8. Sobre o item 3 do tópico ATENÇÃO (VLA): optamos por matrizes alocadas dinamicamente com malloc() (ponteiro para ponteiro) em vez de VLA. Uma VLA é criada na pilha (stack) do programa, que costuma ter apenas alguns MB (cerca de 1 MB no Windows e 8 MB no Linux). Os tamanhos pedidos nos cálculos de tempo não caberiam nela: só na Função 3 ($n = 300$), cada arranjo $n \times n \times n$ ocupa $300^3 * 4 \text{ bytes} = 108 \text{ MB}$, e na Função 5 ($n = 10.000.000$) cada vetor ocupa 40 MB. Com malloc() a memória vem do heap, que não tem essa limitação, então o programa funciona com qualquer dimensão informada.